DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK

WEEK - 6

NAME: Konda Bhavani

ROLL NUMBER: CH.SC.U4CSE24120

CLASS: CSE-B

Question: Using the Quick Select technique, find the k-th smallest element in the following list. Assume the last element as the pivot in each partition.

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

CODE:

```c
//CH.SC.U4CSE24120
#include <stdio.h>

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low;

    for(int j = low; j < high; j++) {
        if(arr[j] < pivot) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            i++;
        }
    }

    int temp = arr[i];
    arr[i] = arr[high];
    arr[high] = temp;

    return i;
}

int quickSelect(int arr[], int low, int high, int k) {
    if(low <= high) {
        int pivotIndex = partition(arr, low, high);

        if(pivotIndex == k)
            return arr[pivotIndex];
        else if(pivotIndex > k)
            return quickSelect(arr, low, pivotIndex - 1, k);
        else
            return quickSelect(arr, pivotIndex + 1, high, k);
    }
    return -1;
}
```

```c
int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k;
    printf("Enter k: ");
    scanf("%d", &k);

    if(k < 1 || k > n) {
        printf("Invalid k\n");
        return 0;
    }

    int result = quickSelect(arr, 0, n - 1, k - 1);

    printf("k-th smallest element: %d\n", result);

    return 0;
}
```

OUTPUT:

```
Enter k: 3
k-th smallest element: 112
```

WORKING:

Quick Select

157 110 147 122 111 149 151 141 123 112 117 133

$k = 3$ (Index = 2)

Pivot index > required element index
↳ go to left subarray

Pivot index < required element index
↳ go to right subarray

Step 1:- Pivot = 157

Partition is done as: Elements < 157 to left, Elements > 157 to right

After Partition:-

110 147 122 111 149 151 141 123 112 117 133 157

Pivot index = 11 > 2 → go to left sub array.

Step 2:-

Subarray :- 110 147 122 111 149 151 141 123 112 117 133

Choosing new pivot :- 110

After partition:- 110 147 122 111 149 151 141 123 112 117 133

After arranging:- 110 111 122 147 149 151 141 123 112 117 133

Pivot index = 0 < 2 → goto right subarray.

Step 3:-

Subarray:- 111 122 147 149 151 141 123 112 117 133

new pivot :- 111

After partition:- 111 122 147 149 151 141 123 112 117 133

Pivot index = 0 < 2 → go to right subarray.

Step 4:-

Subarray:- 122 147 149 151 141 123 112 117 133

new pivot:- 122

After partition:- 112, 117 122 149 151 141 123 147 133.

Pivot index = 2 = 2

~~Actual~~ final array:- 110 111 112 117 122 149 151 141 123 147 133

From this the 3rd smallest ~~numb~~ element is 112.

Time Complexity:
$O(N^2)$: Average case $O(N)$, but worst case $O(N^2)$ — because each call partitions the array like QuickSort, and in the worst case the pivot always splits poorly.

Space Complexity:
$O(N)$: $O(1)$ auxiliary space (in-place), but $O(N)$ worst-case recursion stack and $O(\log N)$ average recursion depth.